

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Jerry Lu

ID: jl3498

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `c:\Users\shini\hw4-jl3498-hw4`

```

using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned}
 & \max_{x,y} && 3x + 2y \\
 & \text{subject to:} && x + 4y \leq 16 \\
 & && x - 2y \leq -1 \\
 & && 0 \leq x \leq 4 \\
 & && 1 \leq y \leq 4.
 \end{aligned}$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

Problem 2 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```

# Problem 1

# Define the optimization model
model = Model(HiGHS.Optimizer)
@variable(model, 0 <= x <= 4)
@variable(model, 1 <= y <= 4)
@constraint(model, x + 4y <= 16)
@constraint(model, x - 2y <= -1)
@objective(model, Max, 3x + 2y)

# Display the model and solve it
print(model)
optimize!(model)

# Retrieve and print the results
println("Optimal x = ", value(x))
println("Optimal y = ", value(y))
println("Maximum objective value = ", objective_value(model))

```

Max 3 x + 2 y

Subject to

x + 4 y <= 16

x - 2 y <= -1

x >= 0

y >= 1

x <= 4

y <= 4

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 2 rows; 2 cols; 4 nonzeros

Coefficient ranges:

Matrix [1e+00, 4e+00]

Cost [2e+00, 3e+00]

Bound [1e+00, 4e+00]

RHS [1e+00, 2e+01]

Presolving model

2 rows, 2 cols, 4 nonzeros 0s

2 rows, 2 cols, 4 nonzeros 0s

Presolve : Reductions: rows 2(-0); columns 2(-0); elements 4(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Ph1: 0(0)	0s

```

1      1.8000000000e+01 Pr: 0(0) 0s
Model status      : Optimal
Simplex iterations: 1
Objective value    : 1.8000000000e+01
P-D objective error: 0.0000000000e+00
HiGHS run time    : 0.00
Optimal x = 4.0
Optimal y = 3.0
Maximum objective value = 18.0

```

By using the JuMP code for each of the given variables, constraints, and objective, you can solve the x and y values through the `optimize!(model)`.

```

# Problem 2.1 and 2.2

# Data
crops = ["soybean", "wheat", "corn"]
rates = [0, 1, 2]
yield = Dict{
    "soybean" => [2900, 3800, 4400],
    "wheat"   => [3500, 4100, 4200],
    "corn"    => [5900, 6700, 7900],
}
price = Dict("soybean" => 0.36, "wheat" => 0.27, "corn" => 0.22)
prod_cost = Dict("soybean" => 350, "wheat" => 280, "corn" => 390)
pesticide_cost = 70 # $/kg-yr
limits = Dict("soybean" => 0.8, "wheat" => 0.7, "corn" => 0.6)
demand = Dict("soybean" => 250000, "wheat" => 250000, "corn" => 250000)
total_area = 130 # ha

# Compute profit per ha for each crop-rate
profit = Dict{Tuple{String,Int}, Float64}{}
for c in crops, r in rates
    profit[(c, r)] = price[c] * yield[c][r+1] - prod_cost[c] - pesticide_cost * r
end

# Model
model = Model(HiGHS.Optimizer)
@variable(model, x[crops, rates] >= 0) # area (ha)
# Total area constraint
@constraint(model, sum(x) == total_area)
# Demand constraint for each crop
for c in crops

```

```

    @constraint(model, sum(yield[c][r+1] * x[c, r] for r in rates) <= demand[c])
end
# Regulatory pesticide constraints
for c in crops
    @constraint(model, sum((r - limits[c]) * x[c, r] for r in rates) <= 0)
end
# Objective: maximize total profit
@objective(model, Max, sum(profit[(c, r)] * x[c, r] for c in crops, r in rates))
optimize!(model)

# Results
println("Optimal land allocation (ha) and pesticide rates:")
for c in crops
    for r in rates
        val = value(x[c,r])
        if val > 1e-6 # only show non-zero allocations
            println("Crop: $(c), Rate: $(r) kg/ha, Area: ", round(val,digits=2), " ha, Profit: ", round(profit[(c,r)], digits=2))
        end
    end
end
end

println("\nTotal profit = \$", round(objective_value(model), digits=2))

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

6 rows, 8 cols, 28 nonzeros 0s

Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)

Solving the presolved LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	-3.7843715799e+01	Ph1: 6(26.7484); Du: 7(37.8437)	0s
5	-1.1674116702e+05	Pr: 0(0)	0s

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 5

```

Objective value      : 1.1674116702e+05
P-D objective error  : 6.2325284535e-17
HiGHS run time      : 0.00
Optimal land allocation (ha) and pesticide rates:
Crop: soybean, Rate: 0 kg/ha, Area: 13.81 ha, Profit/ha: 694.0
Crop: soybean, Rate: 1 kg/ha, Area: 55.25 ha, Profit/ha: 948.0
Crop: wheat, Rate: 0 kg/ha, Area: 6.74 ha, Profit/ha: 665.00000000000001
Crop: wheat, Rate: 1 kg/ha, Area: 15.73 ha, Profit/ha: 757.0
Crop: corn, Rate: 0 kg/ha, Area: 26.92 ha, Profit/ha: 908.0
Crop: corn, Rate: 2 kg/ha, Area: 11.54 ha, Profit/ha: 1208.0

```

Total profit = \$116741.17

2.1

After inputting the given data from the problem like the data, regularity limits and demand, you solve for the profit per hectare for each crop-rate combination. Then a model like problem one was used for each of the variables, constraints, and objective. The objective found the total profit. Each of the land, demand, and pesticide constraints were given and optimized (model) found the optimal allocation.

```

# Problem 2.3

# Data
crops = ["soybean", "wheat", "corn"]
rates = [0, 1, 2]
yield = Dict{
    "soybean" => [2900, 3800, 4400],
    "wheat"   => [3500, 4100, 4200],
    "corn"    => [5900, 6700, 7900],
}
price = Dict{String => 0.36, "wheat" => 0.27, "corn" => 0.22}
prod_cost = Dict{String => 350, "wheat" => 280, "corn" => 390}
pesticide_cost = 70 # $/kg-yr
limits = Dict{String => 0.8, "wheat" => 0.7, "corn" => 0.6}
demand = Dict{String => 250000, "wheat" => 250000, "corn" => 250000}
total_area = 140 # ha

# Compute profit per ha for each crop-rate
profit = Dict{Tuple{String, Int}, Float64}()
for c in crops, r in rates
    profit[(c, r)] = price[c] * yield[c][r+1] - prod_cost[c] - pesticide_cost * r
end

```

```

# Model
model = Model(HiGHS.Optimizer)
@variable(model, x[crops, rates] >= 0) # area (ha)
# Total area constraint
@constraint(model, sum(x) == total_area)
# Demand constraint for each crop
for c in crops
    @constraint(model, sum(yield[c][r+1] * x[c, r] for r in rates) <= demand[c])
end
# Regulatory pesticide constraints
for c in crops
    @constraint(model, sum((r - limits[c]) * x[c, r] for r in rates) <= 0)
end
# Objective: maximize total profit
@objective(model, Max, sum(profit[(c, r)] * x[c, r] for c in crops, r in rates))
optimize!(model)

# Results
println("Optimal land allocation (ha) and pesticide rates:")
for c in crops
    for r in rates
        val = value(x[c,r])
        if val > 1e-6 # only show non-zero allocations
            println("Crop: $(c), Rate: $(r) kg/ha, Area: ", round(val,digits=2), " ha, Profit: ")
        end
    end
end
println("\nTotal profit = \$", round(objective_value(model), digits=2))

```

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
LP   has 7 rows; 9 cols; 27 nonzeros
Coefficient ranges:
  Matrix [2e-01, 8e+03]
   Cost  [7e+02, 1e+03]
  Bound  [0e+00, 0e+00]
   RHS   [1e+02, 2e+05]
Presolving model
7 rows, 9 cols, 27 nonzeros  0s
6 rows, 8 cols, 28 nonzeros  0s

```



```

Presolve : Reductions: rows 6(-1); columns 8(-1); elements 28(+1)
Solving the presolved LP
Using EKK dual simplex solver - serial
  Iteration      Objective      Infeasibilities num(sum)
      0      -3.7843715799e+01 Ph1: 6(26.7484); Du: 7(37.8437) 0s
      5      -1.2403516702e+05 Pr: 0(0) 0s
Solving the original LP from the solution after postsolve
Model status      : Optimal
Simplex iterations: 5
Objective value    : 1.2403516702e+05
P-D objective error : 5.8660204673e-17
HiGHS run time     : 0.00
Optimal land allocation (ha) and pesticide rates:
Crop: soybean, Rate: 0 kg/ha, Area: 13.81 ha, Profit/ha: 694.0
Crop: soybean, Rate: 1 kg/ha, Area: 55.25 ha, Profit/ha: 948.0
Crop: wheat, Rate: 0 kg/ha, Area: 9.74 ha, Profit/ha: 665.0000000000001
Crop: wheat, Rate: 1 kg/ha, Area: 22.73 ha, Profit/ha: 757.0
Crop: corn, Rate: 0 kg/ha, Area: 26.92 ha, Profit/ha: 908.0
Crop: corn, Rate: 2 kg/ha, Area: 11.54 ha, Profit/ha: 1208.0

```

Total profit = \$124035.17

2.3

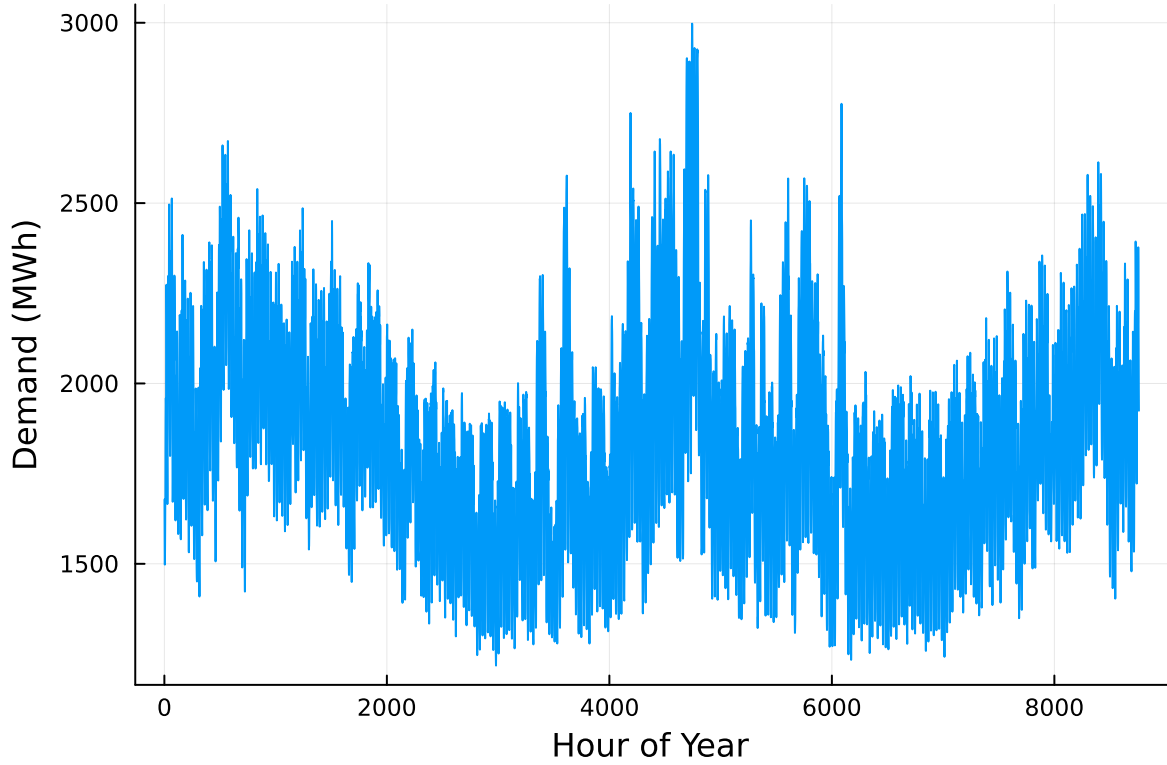
The extra 10 ha increased the profit to \$124035.17 from \$116741.17 which is \$7294 profit. This value makes sense since the land before was fully used and this extra 10 ha doesn't exceed the regulatory limit or demand. It is recommended for the farmers to buy when the cost per per ha is less than the marginal profit per ha and the limits and demands are met.

```

# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :Time Stamp => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)

```



Next, we load the generator data, shown in Table 3. This data includes fixed costs (\$/MW installed), variable costs (\$/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

```

# load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)

```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO₂ limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO₂/yr.

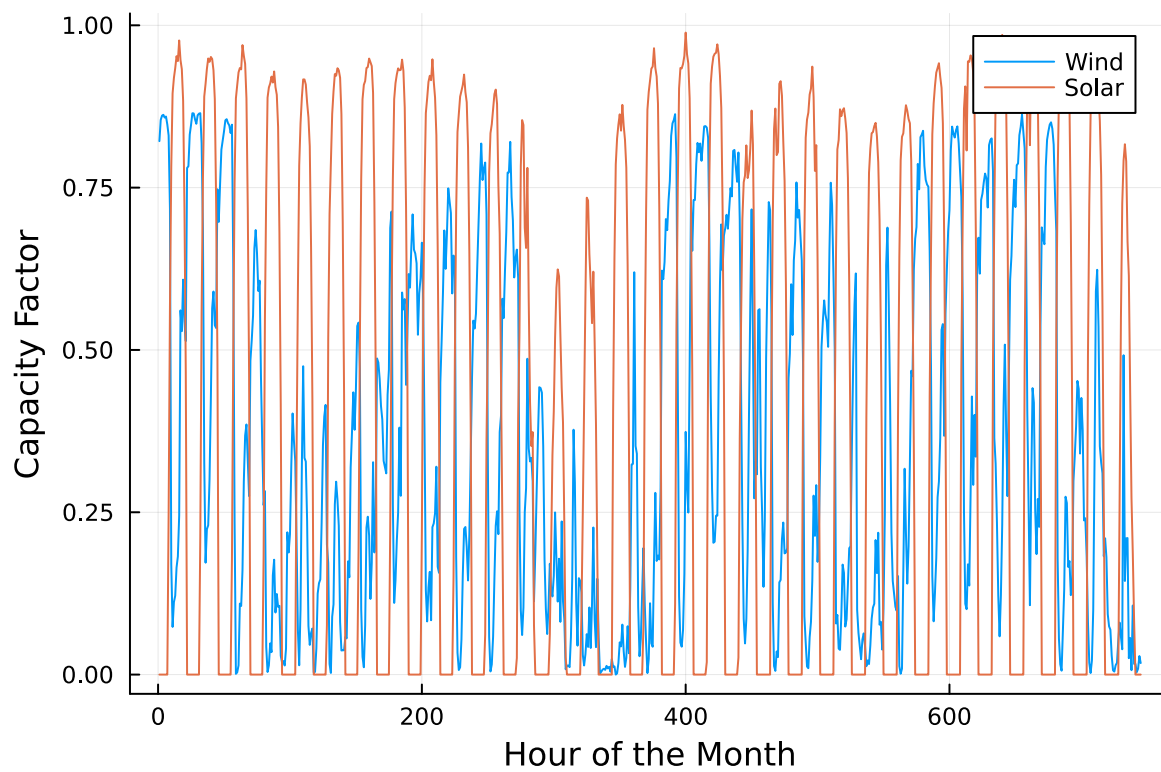
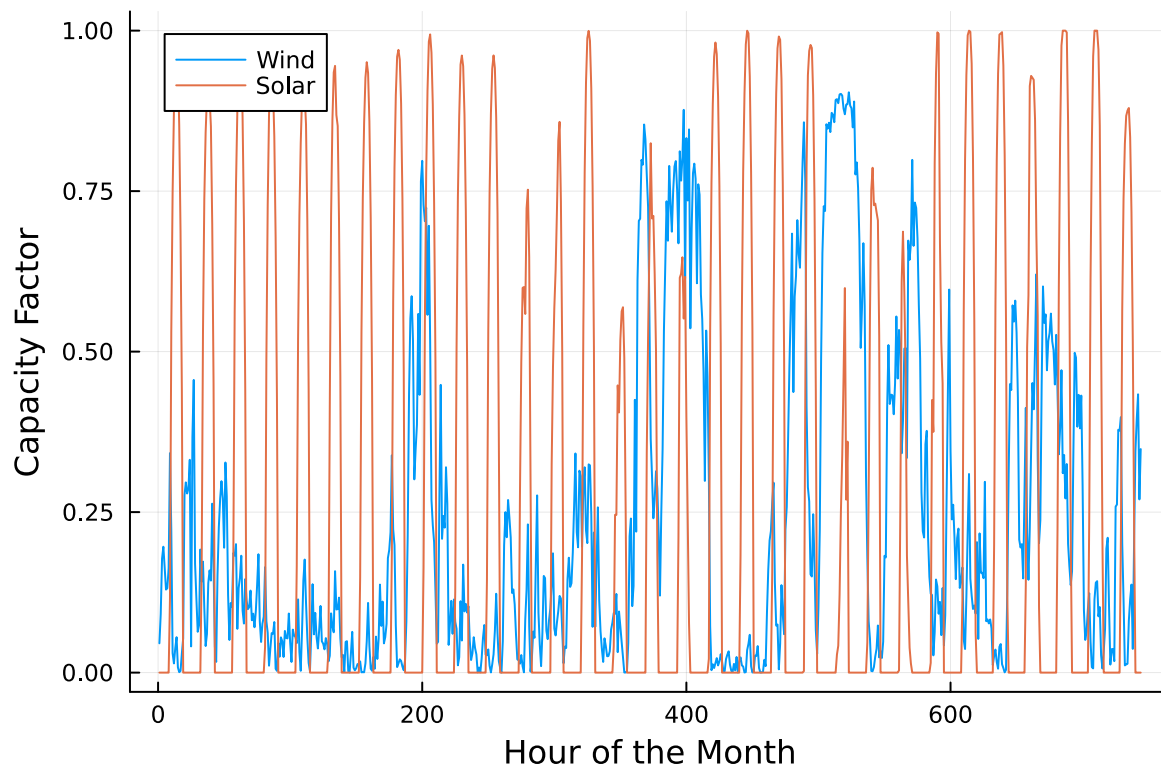
While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?



Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO₂? What is the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

```
# Problem 3.1

# load demand, generators, and capacity factors data
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :Time Stamp => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)
H = 1:nrow(demand) # all hours
```

```

gens = DataFrame(CSV.File("data/generators.csv"))
G = gens.Plant # generator types
CapCost = Dict(gens.Plant .=> gens.FixedCost) # $/MW installed
VarCost = Dict(gens.Plant .=> gens.VarCost) # $/MWh
EF = Dict(gens.Plant .=> gens.Emissions) # tCO2/MWh
CF_dispatchable = Dict("Coal"=>1.0, "NG CCGT"=>1.0, "NG CT"=>1.0, "Geothermal"=>0.85)
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))
CF_hourly = Dict{Tuple{String,Int},Float64}()

for (i,h) in enumerate(H)
    CF_hourly[("Solar", h)] = cap_factor.Solar[i]
    CF_hourly[("Wind", h)] = cap_factor.Wind[i]
end

# Model
model = Model(HiGHS.Optimizer)
# Decision variables
@variable(model, Cap[g in G] >= 0) # Installed capacity (MW)
@variable(model, Gen[g in G, h in H] >= 0) # Hourly generation (MW)
@variable(model, Unserved[h in H] >= 0) # Unserved load (MW)
# Load balance
for h in H
    @constraint(model, sum(Gen[g,h] for g in G) + Unserved[h] == demand.Demand[h])
end
# Hourly capacity limits
for g in G, h in H
    if g in keys(CF_dispatchable)
        cf = CF_dispatchable[g]
    else
        cf = CF_hourly[(g,h)]
    end
    @constraint(model, Gen[g,h] <= Cap[g] * cf)
end
# CO2 limit (fossil generators only)
fossil = ["Coal", "NG CCGT", "NG CT"]
CO2Limit = 1.5e6 # tCO2/year
@constraint(model, sum(Gen[g,h] * EF[g] for g in fossil, h in H) <= CO2Limit)

# Objective: minimize total cost
Penalty_Unserved = 10_000 # $/MWh
@objective(model, Min,
    sum(CapCost[g]*Cap[g] for g in G) +

```

```

        sum(VarCost[g]*Gen[g,h] for g in G, h in H) +
        sum(Penalty_Unserved*Unserved[h] for h in H))
optimize!(model)

# Results
println("Optimal installed capacities (MW) for each generator:")
for g in G
    println("$g: ", round(value(Cap[g]), digits=2), " MW")
end

# Total unserved energy over the year (MWh)
total_unserved = sum(value.(Unserved))
println("\nTotal unserved energy (MWh): ", round(total_unserved, digits=2))

# Total cost
total_cost = objective_value(model)
println("Total cost: \$", round(total_cost, digits=2))

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms
 LP has 61321 rows; 61326 cols; 188256 nonzeros

Coefficient ranges:

```

Matrix [5e-05, 1e+00]
Cost    [2e+01, 4e+05]
Bound   [0e+00, 0e+00]
RHS     [1e+03, 2e+06]

```

Presolving model

56857 rows, 56862 cols, 179328 nonzeros 0s

Dependent equations search running on 8760 equations with time limit of 1000.00s

Dependent equations search removed 0 rows and 0 nonzeros in 0.00s (limit = 1000.00s)

56857 rows, 56862 cols, 179328 nonzeros 0s

Presolve : Reductions: rows 56857(-4464); columns 56862(-4464); elements 179328(-8928)

Solving the presolved LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	0.0000000000e+00	Pr: 8760(5.18855e+06)	0s
22568	5.8499378472e+08	Pr: 10118(4.80812e+06); Du: 0(1.41901e-07)	6s
28647	7.5275118118e+08	Pr: 8637(2.20037e+06); Du: 0(7.07016e-08)	11s
31639	7.8397942134e+08	Pr: 653(21310.8); Du: 0(7.07016e-08)	16s
32698	7.8535295894e+08	Pr: 0(0)	18s

Solving the original LP from the solution after postsolve

Model status : Optimal

Simplex iterations: 32698

```

Objective value      : 7.8535295894e+08
P-D objective error  : 1.3357583179e-14
HiGHS run time      : 17.85
Optimal installed capacities (MW) for each generator:
Geothermal: 285.57 MW
Coal: 0.0 MW
NG CCGT: 1509.17 MW
NG CT: 703.06 MW
Wind: 2548.89 MW
Solar: 2103.75 MW

Total unserved energy (MWh): 332.29
Total cost: $7.8535295894e8

```

3.1

This code uses the parameters for the loading data for demand, generators, and capacity factors. Then the JUMP model was used with each of the decision variables, load balance constraints, hourly capacity constraints, and CO2 emissions limit. The objective function finds the total costs and optimize!(model) finds the optimal number of installed capacity and hourly generation.

3.2

The total amount for each type of generating plant, total cost, and total energy is in problem 3.1

```

# Problem 3.3

# Annual generation per plant
annual_generation = Dict{String,Float64}{}

for g in G
    annual_generation[g] = sum(value.(Gen[g,h]) for h in H)
end

# Total annual generation
total_generation = sum(values(annual_generation))

#Fraction of annual generation
println("Fraction of annual generation by plant type:")
for g in G
    fraction = annual_generation[g] / total_generation
    println("$g: ", round(fraction*100, digits=2), "%")
end

```



```

end

# Installed capacity vs fraction of generation
println("\nInstalled capacity (MW) vs fraction of generation:")
for g in G
    cap_fraction = value(Cap[g]) / sum(value.(Cap))
    gen_fraction = annual_generation[g] / total_generation
    println("$g: Capacity fraction = ", round(cap_fraction*100,digits=2),
           "%, Generation fraction = ", round(gen_fraction*100,digits=2), "%")
end

```

Fraction of annual generation by plant type:

Geothermal: 6.7%

Coal: 0.0%

NG CCGT: 20.3%

NG CT: 0.79%

Wind: 39.42%

Solar: 32.79%

Installed capacity (MW) vs fraction of generation:

Geothermal: Capacity fraction = 3.99%, Generation fraction = 6.7%

Coal: Capacity fraction = 0.0%, Generation fraction = 0.0%

NG CCGT: Capacity fraction = 21.11%, Generation fraction = 20.3%

NG CT: Capacity fraction = 9.83%, Generation fraction = 0.79%

Wind: Capacity fraction = 35.65%, Generation fraction = 39.42%

Solar: Capacity fraction = 29.42%, Generation fraction = 32.79%

3.3

This code finds the total annual generation and the fraction of the annual/total generation. Wind and solar have higher generation fraction and installed capacity making them cheap and emission free. Dispatchable plants like Geothermal, NG CCGT, and NG CT provide reliability with NG CT being used during high demand. Coal is zero due to CO2 cap and costs. This makes sense given the demand and generator data.

Problem 3.5

```

CO2Constraint = @constraint(model, sum(Gen[g,h] * EF[g] for g in fossil, h in H) <= CO2Limit)
optimize!(model)

```

Get shadow price of CO2 constraint

```

co2_dual = shadow_price(CO2Constraint) # get shadow price of CO2 constraint

```

```
println("Shadow price of CO2: \$", round(co2_dual, digits=2), " per tCO2")

# Value to utility of relaxing CO2 limit by 1000 tCO2
additional_CO2 = 1000 # tCO2
value_relax = co2_dual * additional_CO2
println("Value to utility of allowing 1000 tCO2 more: \$", round(value_relax, digits=2))
```

LP has 61322 rows; 61326 cols; 214536 nonzeros

Coefficient ranges:

Matrix [5e-05, 1e+00]

Cost [2e+01, 4e+05]

Bound [0e+00, 0e+00]

RHS [1e+03, 2e+06]

Solving LP without presolve, or with basis, or unconstrained

Model status : Optimal

Objective value : 7.8535295894e+08

P-D objective error : 1.3357583179e-14

HiGHS run time : 0.11

Shadow price of CO2: \$0.0 per tCO2

Value to utility of allowing 1000 tCO2 more: \$0.0

References

List any external references consulted, including classmates.